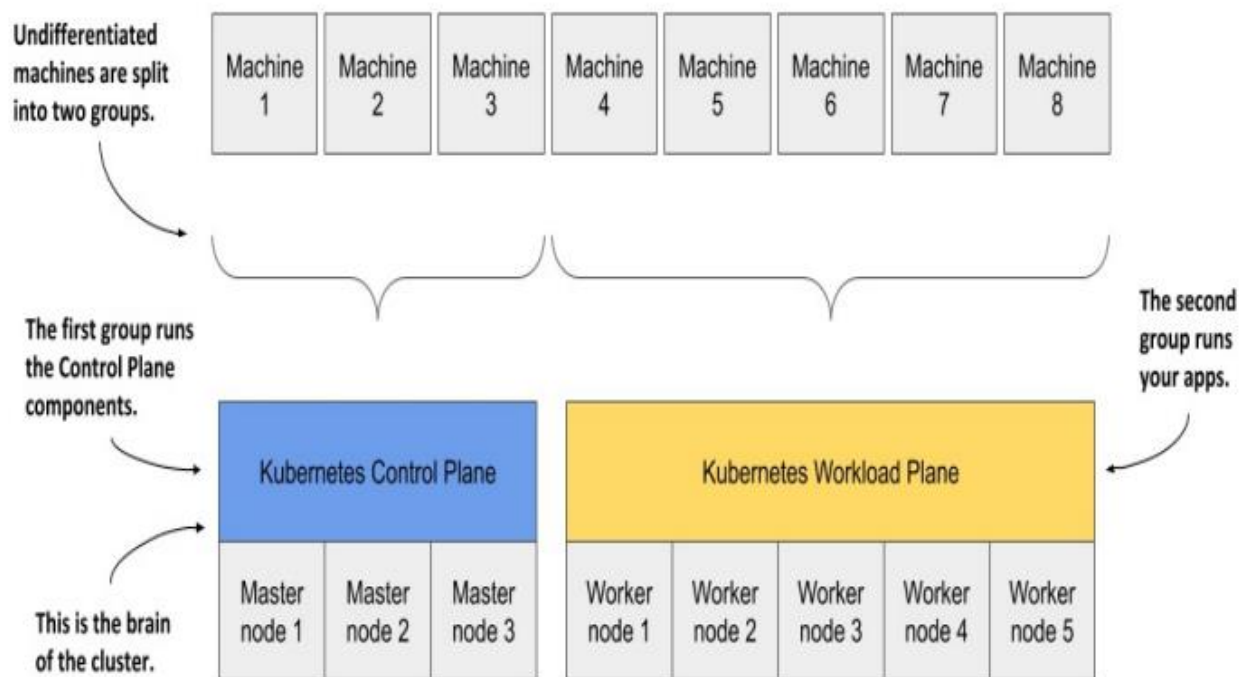


As soon as you deploy an application to Kubernetes, it takes over **the daily management of the application**. If the application fails, Kubernetes **will automatically restart it**. If the hardware fails or the **infrastructure topology changes** so that **the application needs to be moved to other machines**, Kubernetes does this all by itself. The engineers responsible for operating the system can focus on the big picture instead of wasting time on the details.

=====

It frees application developers from the need to implement infrastructure, related mechanisms in their applications; instead, they rely on Kubernetes to provide them. This includes things like:

- **service discovery** - a mechanism that allows applications to find other applications and use the services they provide.
 - **horizontal scaling** - replicating your application to adjust to fluctuations in load.
 - **load-balancing** - distributing load across all the application replicas.
 - **self-healing** - keeping the system healthy by automatically restarting failed applications and moving them to healthy nodes after their nodes fail.
 - **leader election** - a mechanism that decides which instance of the application should be active while the others remain idle but ready to take over if the active instance fails.
- =====

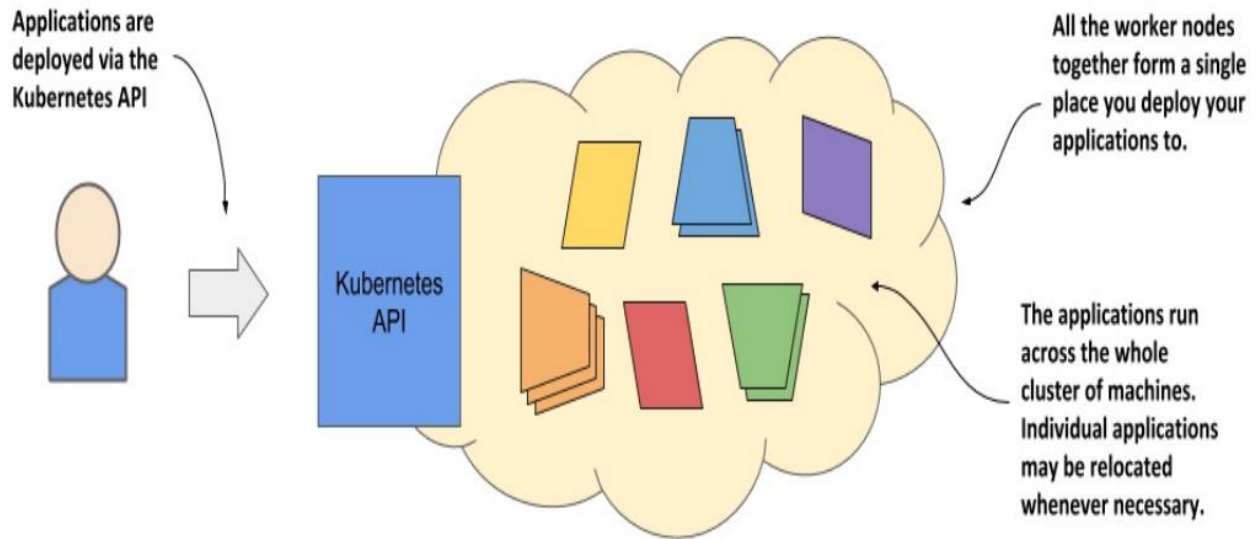


The master nodes will **run the Kubernetes Control Plane**, which represents the brain of your system and **controls the cluster**, while the rest will run your applications - your workloads - and will therefore represent the Workload Plane.

The **Workload Plane** is sometimes referred to as the **Data Plane**, but this term could be confusing because the plane **doesn't host data but applications**. Don't be confused by the term "plane" either - in this context you can think of it as the "**surface**" the applications run on.

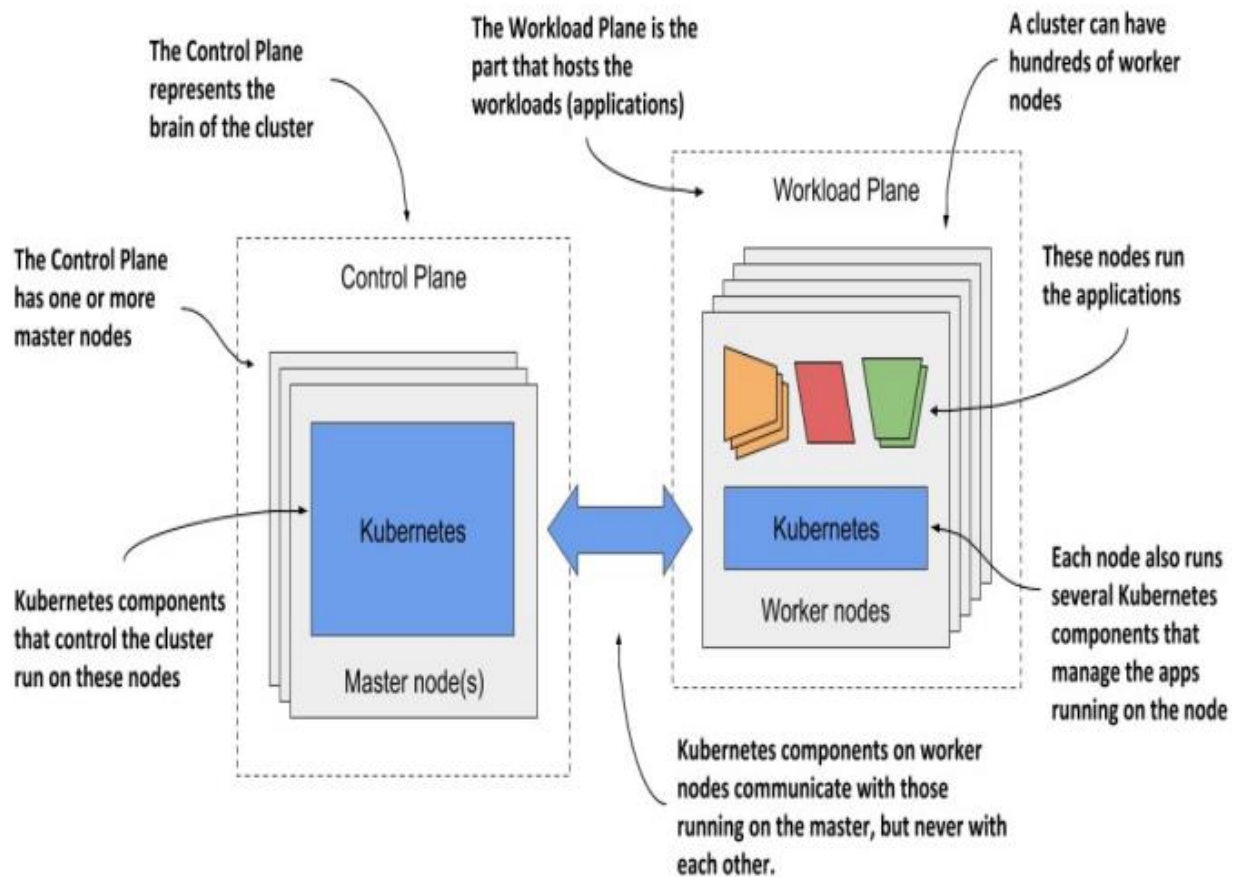
Non-production clusters can use a single master node, but highly available clusters use **at least three physical master nodes to host the Control Plane**. The number of worker nodes depends on the number of applications you'll deploy.

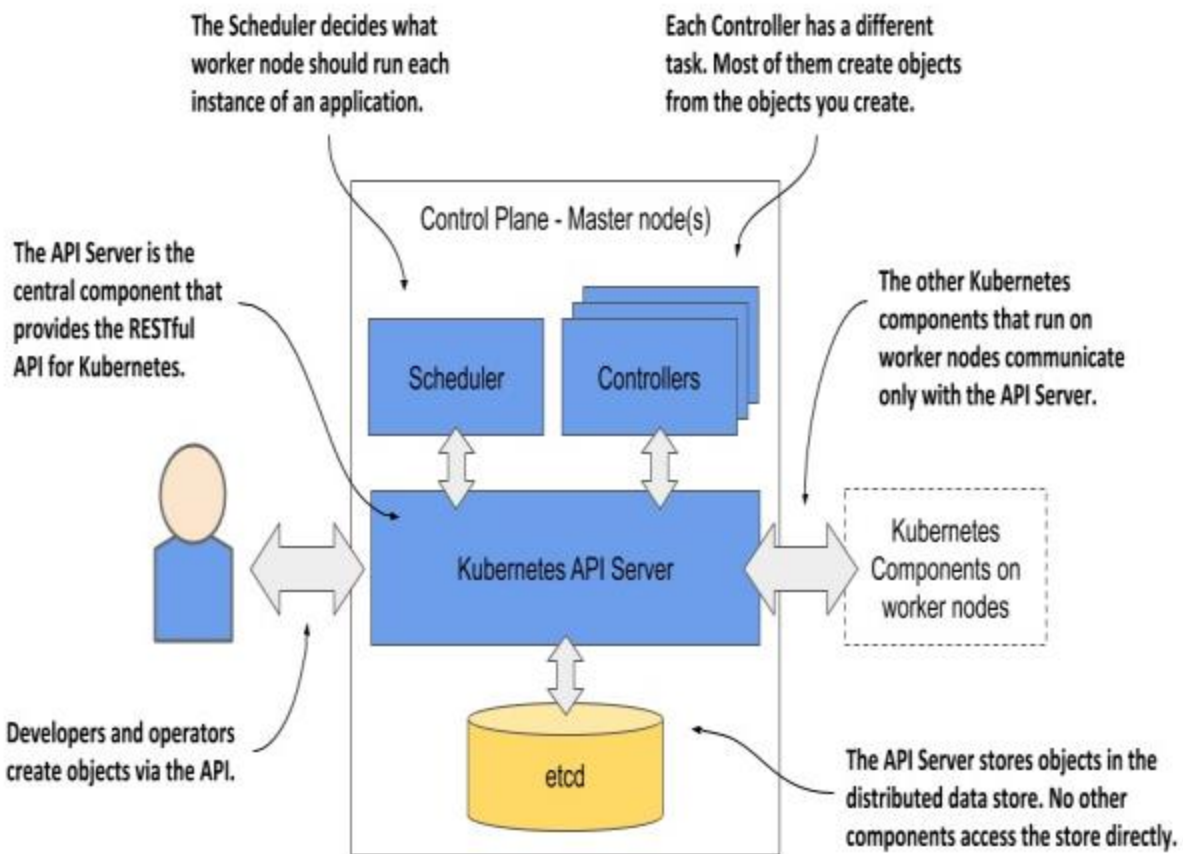
Figure 1.10 Kubernetes exposes the cluster as a uniform deployment area



Kubernetes is a self-healing system in that it deals with software errors like the one just described, but it also **handles hardware failures**. As clusters grow in size, the frequency of node failure also increases. For example, in a cluster with one hundred nodes and a MTBF (**mean-time-between-failure**) of 100 days for each node, you can expect one node to fail every day.

Figure 1.11 The two planes that make up a Kubernetes cluster





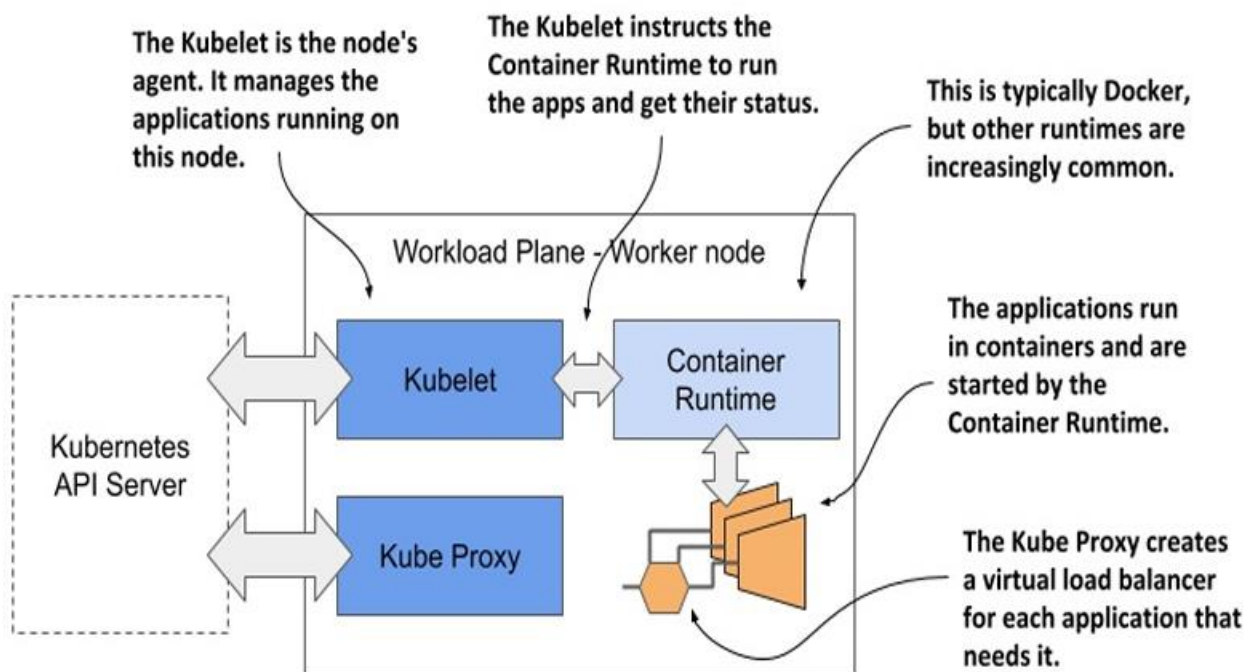
These are the components and their functions:

- **The Kubernetes API Server** exposes the RESTful Kubernetes API. Engineers using the cluster and other Kubernetes components create objects via this API.
- **The etcd distributed datastore** persists the objects you create through the API, since the API Server itself is stateless. The Server is the only component that talks to etcd.
- **The Scheduler** decides on which worker node each application instance should run.
- **Controllers** bring to life the objects you create through the API. Most of them simply create other objects, but some also communicate with external systems (for example, the cloud provider via its API).

The components of the Control Plane hold and control the state of the cluster, but they don't run your applications. **This is done by the (worker) nodes.**

Worker node components:

The worker nodes are the computers on which your applications run. **They form the cluster's Workload Plane**. In addition to applications, several Kubernetes components also run on these nodes. They perform the task of **running, monitoring** and **providing connectivity** between your applications.



Each node runs the following set of components:

- The **Kubelet**, an agent that talks to the API server and manages the applications running on its node. It reports the status of these applications and the node via the API.
- The **Container Runtime**, which can be Docker or any other runtime compatible with Kubernetes. It runs your applications in containers as instructed by the Kubelet.
- The **Kubernetes Service Proxy (Kube Proxy)** load-balances network traffic between applications. Its name suggests that traffic flows through it, but that's no longer the case.

=====

Add-on components

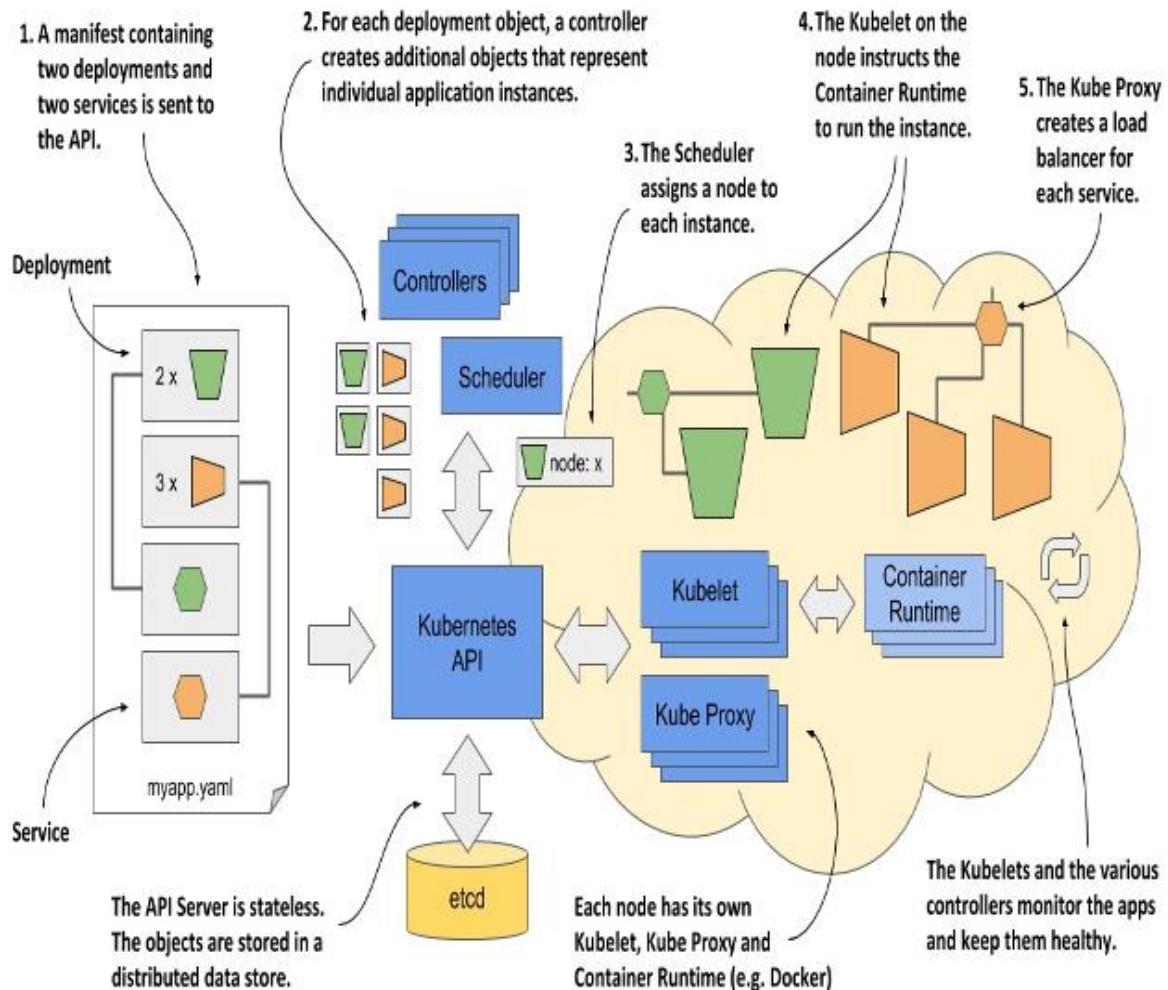
Most Kubernetes clusters also contain several other components. This includes a DNS server, network plugins, logging agents and many others. They typically run on the worker nodes but can also be configured to run on the master.

=====

Defining your application

Everything in Kubernetes is represented **by an object**. You create and retrieve these objects via the Kubernetes API. Your application consists of several types of these objects - one type represents the application deployment as a whole, another represents a running instance of your application, another represents the service provided by a set of these instances and allows reaching them at a single IP address, and there are many others.

=====



The top managed Kubernetes offerings include the following:

- Google Kubernetes Engine (GKE)
- Azure Kubernetes Service (AKS)
- Amazon Elastic Kubernetes Service (EKS)
- IBM Cloud Kubernetes Service
- Red Hat OpenShift Online and Dedicated
- VMware Cloud PKS
- Alibaba Cloud Container Service for Kubernetes (ACK)

1.3.3 Using vanilla or extended Kubernetes

The final question is whether to use a vanilla open-source version of Kubernetes or an extended, enterprise-quality Kubernetes product.

Using a vanilla version of Kubernetes

The open-source version of Kubernetes is maintained by the community and represents the cutting edge of Kubernetes development. This also means that it may not be as stable as the other options. It may also lack good security defaults. Deploying the vanilla version requires a lot of fine tuning to set

everything up for production use.

Using enterprise-grade Kubernetes distributions

A better option for using Kubernetes in production is to use an enterprise-quality Kubernetes distribution such as OpenShift or Rancher. In addition to the increased security and performance provided by better defaults, they offer additional object types in addition to those provided in the upstream Kubernetes API. For example, vanilla Kubernetes does not contain object types that represent cluster users, whereas commercial distributions do. They also provide additional software tools for deploying and managing well-known third-party applications on Kubernetes.

Of course, extending and hardening Kubernetes takes time, so these commercial Kubernetes distributions usually lag one or two versions behind the upstream version of Kubernetes. It's not as bad as it sounds. The benefits usually outweigh the disadvantages.

=====